

Smart Home Sensor Monitoring & Control System (Socket Programming)

Course: Computer Networks

Name: Zainab Khan

Date: October 29th 2025

Objective

The goal of this assignment is to design and implement a Smart Home Monitoring & Control System that demonstrates the difference between reliable TCP communication (for control commands) and fast UDP communication (for real-time sensor data).

System Architecture

The Smart Hub (server) and IoT Devices (clients) communicate using both TCP and UDP.

- TCP Server (Port 5050): handles device registration and sends reliable control commands.
- UDP Server (Port 6060): receives fast sensor data streams.
- Clients: register over TCP and stream readings over UDP.

TCP Implementation – Device Control & Registration

Server Highlights

- Accepts multiple device connections (threaded).
- Stores device ID and type in a dictionary.
- Allows interactive commands: list, send, broadcast, quit.
- Logs all activity to server_log.txt.

Client Highlights

- Registers with DEVICE <name> TYPE <type>.
- Waits for commands and simulates execution.
- Sends ACK Command Executed.

```
list
- Sensor01 (type=temperature, addr=('127.0.0.1', 51407))
```

Figure 1: TCP Server showing registered device.

```

hub> send Sensor01 SET_INTERVAL 3
[2025-10-29 19:48:37] [Server] Sent to Sensor01: 'SET_INTERVAL 3'
[2025-10-29 19:48:37] [Server] From Sensor01: ACK Command Executed
hub> send Sensor01 ACTIVATE_ALARM
[2025-10-29 19:48:40] [Server] Sent to Sensor01: 'ACTIVATE_ALARM'
[2025-10-29 19:48:40] [Server] From Sensor01: ACK Command Executed
hub>

```

Figure 2: Server sending commands and receiving ACKs.

```

[Sensor01] Sent registration: DEVICE Sensor01 TYPE temperature
[Sensor01] Received command: 'SET_INTERVAL 3'
[Sensor01] Sensor01: interval set to 3s
[Sensor01] Sent ACK
[Sensor01] Received command: 'ACTIVATE_ALARM'
[Sensor01] Sensor01: Alarm activated
[Sensor01] Sent ACK

```

Figure 3: Client executing commands and sending ACKs.

Sample log snippet (from server_log.txt):

```

[2025-10-29 19:48:01] [Server] Listening on 127.0.0.1:5050
[2025-10-29 19:48:18] [Server] Connected to ('127.0.0.1', 51407)
[2025-10-29 19:48:18] [Server] Registered DEVICE=Sensor01 TYPE=temperature
[2025-10-29 19:48:37] [Server] Sent to Sensor01: 'SET_INTERVAL 3'
[2025-10-29 19:48:37] [Server] From Sensor01: ACK Command Executed
[2025-10-29 19:48:40] [Server] Sent to Sensor01: 'ACTIVATE_ALARM'
[2025-10-29 19:48:40] [Server] From Sensor01: ACK Command Executed
[2025-10-29 19:53:18] [Server] Shutting down on user request.

```

UDP Implementation – Real Time Sensor Streaming

Server Highlights

- Listens on 6060, receives packets like
- Sensor01,2025-10-29 19:36:42,temperature,22.0,SEQ:10
- Tracks sequence numbers 1-10 per device.
- Detects missing packets and sends STATUS RECEIVED X/10 PACKETS.
- Logs data to sensor_data_log.txt.

Client Highlights

- Sends 10 sequenced sensor packets per cycle.
- Waits for STATUS from server (with up to 2 retries on timeout).
- DROP_SEQ variable used to simulate packet loss.

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\zayna> cd $HOME\Desktop\SmartHomeSocket_Zainab
PS C:\Users\zayna\Desktop\SmartHomeSocket_Zainab> .\venv\Scripts\Activate.ps1
(.venv) PS C:\Users\zayna\Desktop\SmartHomeSocket_Zainab> python .\udp_server.py
[2025-10-29 19:55:51] [UDP-Server] Listening on 127.0.0.1:6060
[2025-10-29 19:56:07] [UDP-Server] RX from Sensor01 @ ('127.0.0.1', 54941): ts=2025-10-29 19:56:07, type=temperature, value=23.5, SEQ=1
[2025-10-29 19:56:08] [UDP-Server] RX from Sensor01 @ ('127.0.0.1', 54941): ts=2025-10-29 19:56:08, type=temperature, value=23.8, SEQ=2
[2025-10-29 19:56:09] [UDP-Server] RX from Sensor01 @ ('127.0.0.1', 54941): ts=2025-10-29 19:56:09, type=temperature, value=26.9, SEQ=3
[2025-10-29 19:56:10] [UDP-Server] RX from Sensor01 @ ('127.0.0.1', 54941): ts=2025-10-29 19:56:10, type=temperature, value=25.5, SEQ=4
[2025-10-29 19:56:11] [UDP-Server] RX from Sensor01 @ ('127.0.0.1', 54941): ts=2025-10-29 19:56:11, type=temperature, value=24.2, SEQ=5
[2025-10-29 19:56:12] [UDP-Server] RX from Sensor01 @ ('127.0.0.1', 54941): ts=2025-10-29 19:56:12, type=temperature, value=23.4, SEQ=6
[2025-10-29 19:56:13] [UDP-Server] RX from Sensor01 @ ('127.0.0.1', 54941): ts=2025-10-29 19:56:13, type=temperature, value=26.5, SEQ=7
[2025-10-29 19:56:14] [UDP-Server] RX from Sensor01 @ ('127.0.0.1', 54941): ts=2025-10-29 19:56:14, type=temperature, value=23.6, SEQ=8
[2025-10-29 19:56:15] [UDP-Server] RX from Sensor01 @ ('127.0.0.1', 54941): ts=2025-10-29 19:56:15, type=temperature, value=23.5, SEQ=9
[2025-10-29 19:56:16] [UDP-Server] RX from Sensor01 @ ('127.0.0.1', 54941): ts=2025-10-29 19:56:16, type=temperature, value=24.5, SEQ=10
[2025-10-29 19:56:16] [UDP-Server] Sent to Sensor01: STATUS RECEIVED 10/10 PACKETS

```

Figure 4: UDP Server receiving 10/10 packets and sending status.

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\zayna> cd $HOME\Desktop\SmartHomeSocket_Zainab
PS C:\Users\zayna\Desktop\SmartHomeSocket_Zainab> .\venv\Scripts\Activate.ps1
(.venv) PS C:\Users\zayna\Desktop\SmartHomeSocket_Zainab> python .\udp_client.py
[Sensor01] Sending packet SEQ:1 - temperature=23.5
[Sensor01] Sending packet SEQ:2 - temperature=23.8
[Sensor01] Sending packet SEQ:3 - temperature=26.9
[Sensor01] Sending packet SEQ:4 - temperature=25.5
[Sensor01] Sending packet SEQ:5 - temperature=24.2
[Sensor01] Sending packet SEQ:6 - temperature=23.4
[Sensor01] Sending packet SEQ:7 - temperature=26.5
[Sensor01] Sending packet SEQ:8 - temperature=23.6
[Sensor01] Sending packet SEQ:9 - temperature=23.5
[Sensor01] Sending packet SEQ:10 - temperature=24.5
[Sensor01] STATUS RECEIVED 10/10 PACKETS
(.venv) PS C:\Users\zayna\Desktop\SmartHomeSocket_Zainab>

```

Figure 5: UDP Client receiving status message.

Error Handling & Code Quality

- TCP: handles disconnections, exceptions, and command errors.
- UDP: client retry mechanism and server missing-packet report.
- Logging to text files for traceability.
- Code commented and structured for clarity.

How to Run (Commands for tester)

TCP

```
python tcp_server.py
```

```
python tcp_client.py
```

Server commands: list, send <device> <command>, broadcast <text>, quit

UDP

```
python udp_server.py
```

```
python udp_client.py
```

Set DROP_SEQ = None for normal run; DROP_SEQ = 7 for loss demo.

Conclusion

This project demonstrates how TCP provides reliable, ordered command delivery, while UDP offers lightweight, fast transmission suited for real-time sensor data. Implementing both protocols clarified the trade-offs between reliability and speed in network programming.